

Technical Pipeline Description

The application utilizes a decoupled, three-stage architecture to ensure processing speed, zero-hallucination extraction, and high domain accuracy.

****1. Speech-to-Text Model (whisper-large-v3 via Groq API)****

* ****Role:**** Acts as the acoustic frontend. It rapidly processes the uploaded audio file and generates a verbatim, word-for-word raw transcript. Because it runs on Groq's LPUs, it achieves blistering transcription speeds.

* ****Data Flow:**** Ingests raw audio bytes (.mp3, .wav) and outputs an unstructured, un-diarized Raw Transcript string to Stage 2.

****2. First Language Model - Refinement (gemini-3.5-flash-lite)****

* ****Role:**** Acts as the domain-correction and diarization agent. Because acoustic Whisper struggles with technical acronyms and lacks speaker labels, this LLM analyzes the conversational context to logically inject speaker names (Contextual Diarization). It surgically corrects domain-specific jargon and scrubs conversational filler words ("ums/ahs") while strictly preserving the speaker's original negations, numbers, and intent.

* ****Data Flow:**** Ingests the Raw Transcript and a strict refinement prompt, outputting a highly readable, diarized Refined Transcript string to Stage 3.

****3. Second Language Model - Documentation (gemini-3.5-flash-lite with Pydantic)****

* ****Role:**** Acts as the structured extraction agent. This model is constrained by strict Pydantic schemas (JSON mode) to synthesize the Executive Summary, Key Decisions, and Action Items. It is mathematically forced by the schema rules to output "unspecified" if a task owner or deadline is omitted in the audio, completely eliminating the hallucination risks common in standard summarizers.

* ****Data Flow:**** Ingests the Refined Transcript and outputs a strict Machine-Readable JSON object, which the Streamlit frontend dynamically renders into Markdown cards and PDF exports.